

Clase 146 — Análisis con IDA y Ghidra aplicado a malware

Parte: **6 — Análisis de malware** · Fuente: *Practical Malware Analysis* (Sikorski & Honig) 🕒 Duración estimada: **140 min** · Nivel: **Avanzado**

Objetivo

Usar un desensamblador/decompilador para leer el código del malware y reconstruir su lógica: identificar la función `main`, seguir el grafo de llamadas, reconocer patrones de APIs, renombrar y anotar, y llegar al decompilado en pseudo-C. Ghidra (gratis) e IDA son las herramientas centrales del análisis estático avanzado.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Cargar** una muestra en Ghidra/IDA y navegar el desensamblado y el decompilado.
2. **Localizar** funciones relevantes por strings, imports y referencias cruzadas.
3. **Anotar** el análisis: renombrar funciones/variables y aplicar tipos.
4. **Reconocer** patrones de código (bucles de cifrado, resolución dinámica de APIs, C2).
5. **Extraer** configuración e IOCs directamente del código.

Temas

#	Tema	Por qué importa
1	Carga y auto-análisis	Punto de partida del RE
2	Vistas: disasm, decompiler, graph	Distintos niveles de abstracción
3	Xrefs (referencias cruzadas)	Navegar de un string/API a su uso
4	Renombrado y comentarios	Hace legible un binario hostil
5	Reconocimiento de APIs y patrones	Identifica funcionalidad rápido
6	Resolución dinámica de APIs	Malware oculta imports en runtime
7	Scripting (Ghidra/IDAPython)	Automatiza tareas repetitivas

Definiciones y características

- **Desensamblado:** traducción de bytes a instrucciones ASM. Característica clave: **vista exacta** de la ejecución.
- **Decompilador:** reconstruye pseudo-C desde el ASM. Característica clave: **lectura rápida**, aproximada.

- **Xref:** referencia cruzada a una dirección/símbolo. Característica clave: **navegación por uso**.
- **Resolución dinámica de API:** `LoadLibrary` + `GetProcAddress` en runtime. Característica clave: **oculta capacidades** al análisis estático.
- **API hashing:** resolver APIs por hash del nombre. Característica clave: **evasión** de detección por strings.

Herramientas y preparación

- **Ghidra** (NSA, gratuito) con su decompilador integrado.
- **IDA Free/Pro** con Hex-Rays (si se dispone).
- **x64dbg** para correlacionar estático con dinámico (clase posterior).
- Scripts: **Ghidra Scripts**, **IDAPython**, y plugins como **capa explorer**.

 **Nota ética y de seguridad:** el desensamblado es estático y no ejecuta la muestra, pero cárgala siempre desde la VM aislada. Nunca ejecutes la muestra desde el desensamblador (evita "run/debug" con binarios reales fuera del entorno controlado).

Laboratorio guiado

1. Crea un proyecto en Ghidra e importa la muestra. Deja que el auto-análisis termine.
2. En la ventana de **Symbol Tree/Functions**, localiza el entry point y navega hasta la función que parece `main`.
3. Abre **Defined Strings**. Elige un string revelador (URL, mensaje, clave de registro) y usa **xref** para saltar a la función que lo usa.
4. Estudia esa función en el **decompilador**. Renómbrala (p. ej. `contact_c2`) y comenta el propósito de sus variables.
5. Busca imports sospechosos (`InternetConnect` , `CryptEncrypt` , `CreateRemoteThread`) y desde ellos sigue xrefs a las rutinas que los invocan.
6. Si la IAT está casi vacía, localiza el patrón `GetProcAddress` /API hashing y documenta cómo resuelve funciones.
7. Identifica un posible bucle de descifrado de configuración: reconoce XOR/rotaciones y anota la clave.
8. Exporta un resumen: funciones clave renombradas, capacidades encontradas e IOCs extraídos del código.

Ejercicios

1. Explica la diferencia entre lo que ves en disasm y en el decompilador para una misma función.
2. Usa xrefs para trazar el camino desde un string C2 hasta su envío por red.
3. Reconoce y documenta un algoritmo simple de descifrado (XOR de un byte).
4. Renombra 5 funciones y justifica cada nombre.
5. Escribe un script Ghidra/IDAPython que liste todas las llamadas a `GetProcAddress`.
6. Explica cómo el API hashing dificulta el análisis y cómo lo resolverías.

Reto verificable

Entrega un proyecto de Ghidra **anotado** de una muestra con al menos 5 funciones renombradas, la rutina de C2 identificada y la configuración/IOCs extraídos del código, más un breve informe. **Criterio de aceptación:** otra persona puede abrir el proyecto y, siguiendo tus nombres y comentarios, explicar cómo la muestra contacta su C2 sin partir de cero.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Decompilado incoherente	Muestra empaquetada; desempácala antes de desensamblar
No aparece <code>main</code>	Runtime del compilador; localiza <code>main</code> desde <code>WinMain /CRT</code>
Faltan nombres de API	API hashing o resolución dinámica; reconstruye la tabla
Xrefs vacíos en un string	El string se construye en runtime; búscalo en el decompilado
Perderse en el grafo	Trabaja desde strings/imports hacia dentro, no al revés

Preguntas frecuentes

 **¿Ghidra o IDA?** Ambos son excelentes. Ghidra es gratuito y su decompilador es muy bueno; IDA Pro es el estándar comercial. Aprende el flujo, no solo la herramienta.

 **¿Debo leer todo el binario?** No. El análisis dirigido por objetivos (encontrar C2, cifrado, persistencia) es mucho más eficiente que leer linealmente.

 **¿Cómo trato el API hashing?** Reconoce el algoritmo de hash, precomputa hashes de APIs comunes y mapea; hay scripts públicos que automatizan esto.

Referencias

- Sikorski & Honig, *Practical Malware Analysis*, cap. 5–7.
- Ghidra. <https://ghidra-sre.org>
- Hex-Rays IDA. <https://hex-rays.com>
- Ghidra Scripting API. https://ghidra.re/ghidra_docs/api/

Siguiente clase

Clase 147 - Ofuscacion, packing y unpacking